

Sisteme și algoritmi distribuiți

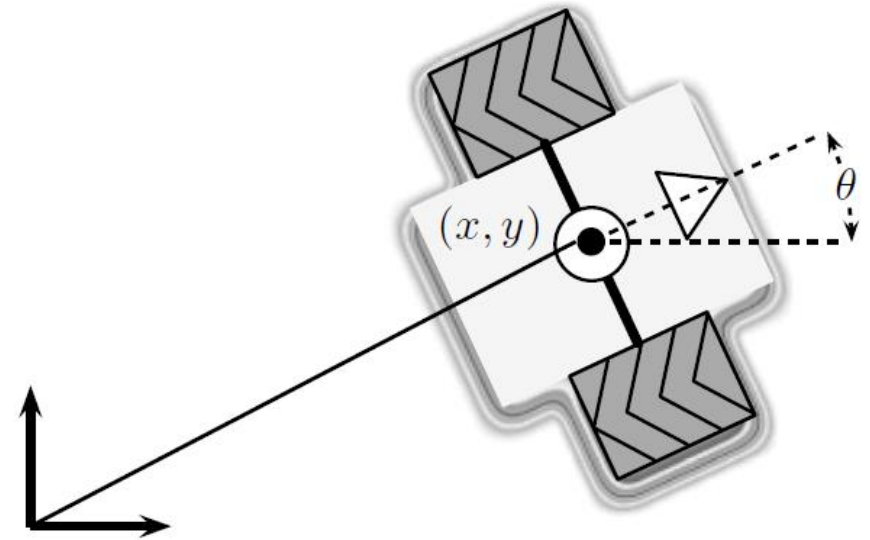
Curs 4

Rendezvous

Problema rendezvous

Un *agent mobil* este un sistem dinamic neliniar (X, U, X_0, f) unde:

- X este spațiul stărilor de dimensiune d
- U este spațiul intrărilor
- $X_0 \subset X$ reprezintă mulțimea stărilor inițiale fezabile
- $f: X \times U \rightarrow R^d$ este operatorul care determină mișcarea agentului prin aplicarea intrării $u(t)$:
$$\underline{x(t+1)} = \underline{f(\underline{x(t)}, \underline{u(t)})}$$



Problema rendezvous

În plan ($d = 2$):

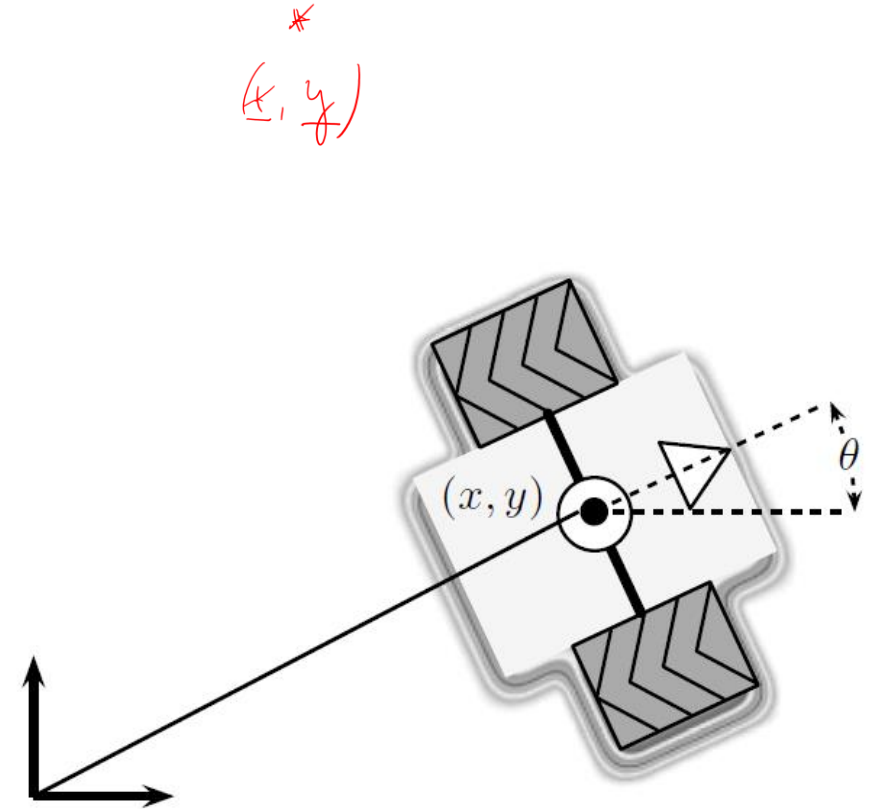
$$\begin{cases} p_x(t+1) = p_x(t) + \underline{v} \cos \theta(t) \\ p_y(t+1) = p_y(t) + v \sin \theta(t) \\ \theta(t+1) = \theta(t) + \underline{\omega} \end{cases}$$

(x, y) poziția planară și θ orientarea unghiulară
(vezi figura alăturată)

$\underline{v}$ și $\underline{\omega}$ sunt mărimile de control care descriu
viteza liniară și cea unghiulară

În acest caz vectorul stării cuprinde:

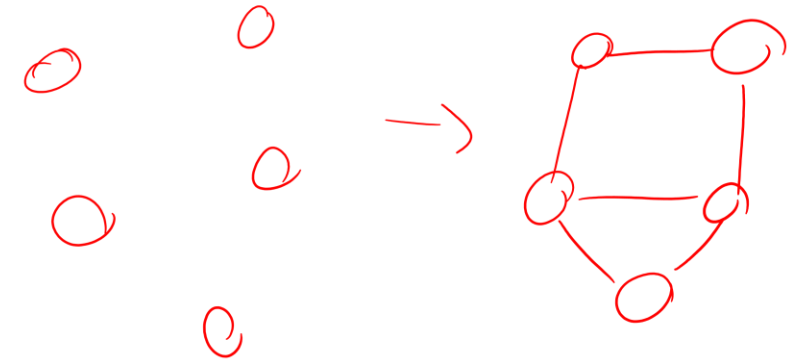
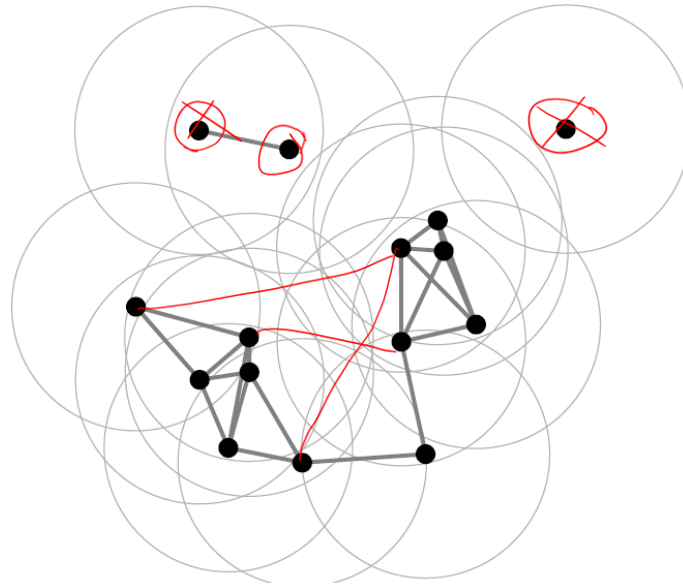
$$x(t) = \begin{bmatrix} p_x(t) \\ p_y(t) \\ \theta(t) \end{bmatrix}$$



Graf de proximitate

Un graf de proximitate $\mathcal{G}$ asociază unui set de puncte distincte $\mathcal{P} = \{p_1, p_2, \dots, p_n\}$ un graf nedirectat, al cărui set de noduri este $\mathcal{P}$, iar muchiile $\mathcal{E}_{\mathcal{G}}(\mathcal{P}) \subseteq \{(p, q) \in \mathcal{P} \times \mathcal{P} \mid p \neq q\}$.

Graful de proximitate r -disc este graful $\mathcal{G}_{disk}(r)$, pentru $r > 0$, în care două noduri sunt legate printr-o muchie dacă distanța dintre ele este cel mult r , i.e. $(p_i, p_j) \in \mathcal{E}_{\mathcal{G}_{disk}}$ dacă $\underbrace{\text{dist}(p_i, p_j)} \leq \underbrace{r}$.



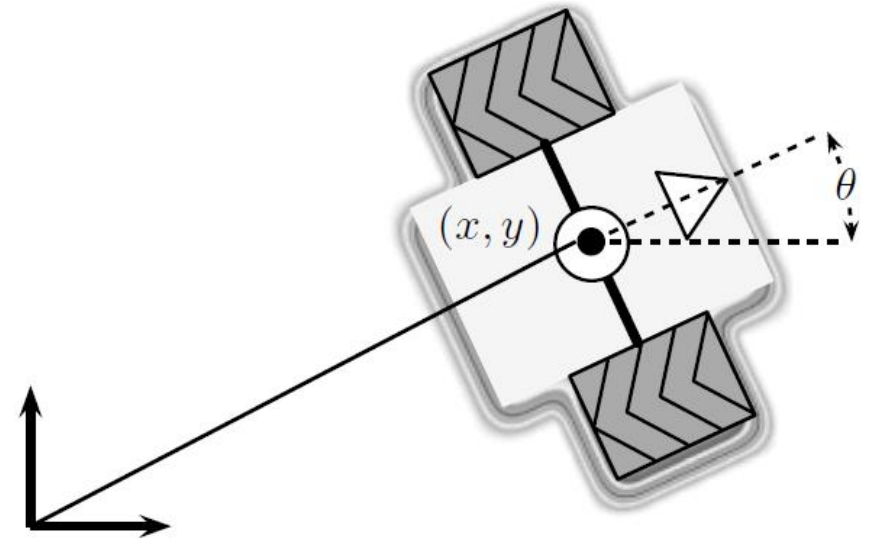
Problema rendezvous

Considerăm o rețea de n agenți mobili $\{1, 2, \dots, n\}$ cu pozițiile inițiale $\{p_1(0), p_2(0), \dots, p_n(0)\}$.

Prin notația $p(t)$ extragem poziția din vectorul de stare $x(t) \in X$, și considerăm modelul simplificat al agenților:

$$p_i(t + 1) = p_i(t) + u_i(t)$$

Problema *rendezvous*: conduceți agenții prin intrări, calculate la nivel local, către un punct de întâlnire arbitrar, i.e. $p_i = p_j, \forall i, j \in \{1, 2, \dots, n\}$.



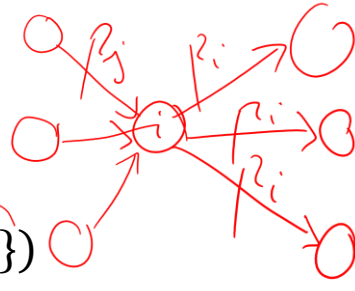
Problema rendezvous

Algoritm **MedieRendezvous()**:

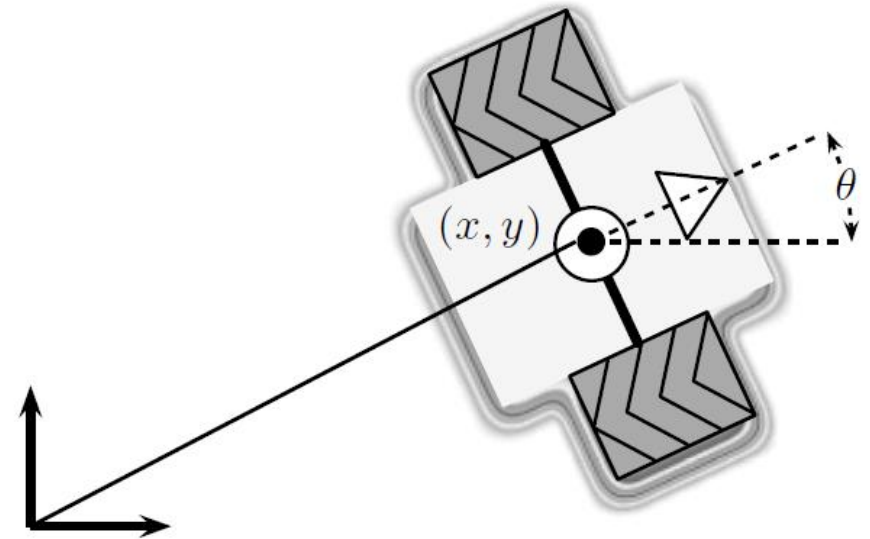
M_i : - int id (id propriu)
- int send_id (var auxiliară), inițial id
- status $\in \{\text{lider, non-lider}\}$, inițial non-lider

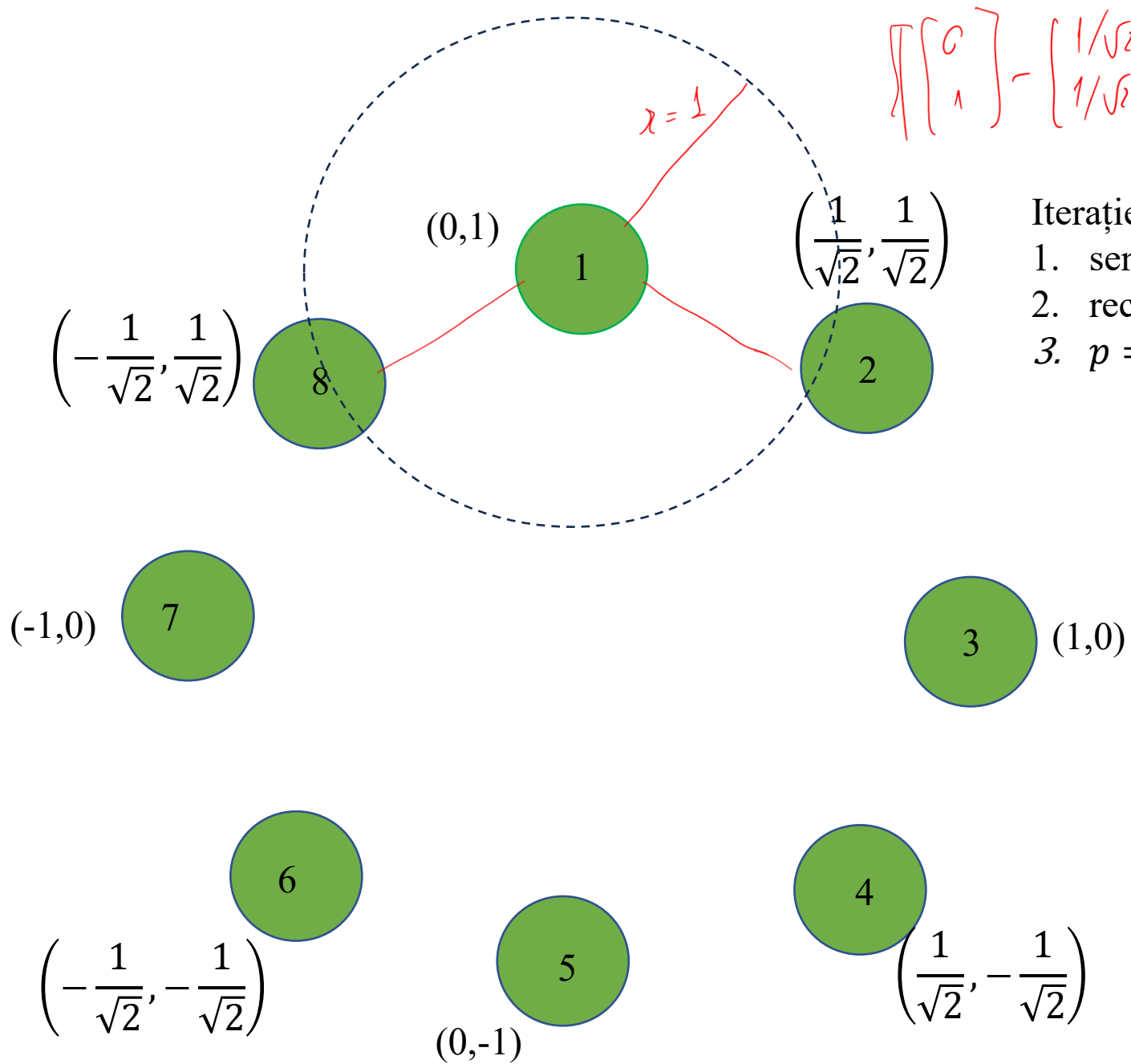
Funcție transformare nod i ():

1. send(p , vecini);
2. recv_p = recv(vecin);
3. p = ~~p~~ avrg($\{p\} \cup \{recv_p\}$)



În fiecare iterație, fiecare agent execută următoarele două operații: difuzează poziția sa și primește pozițiile vecinilor; calculează media mulțimii pozițiilor vecinilor (în uniune cu poziția proprie) pentru a se deplasa într-o nouă poziție.

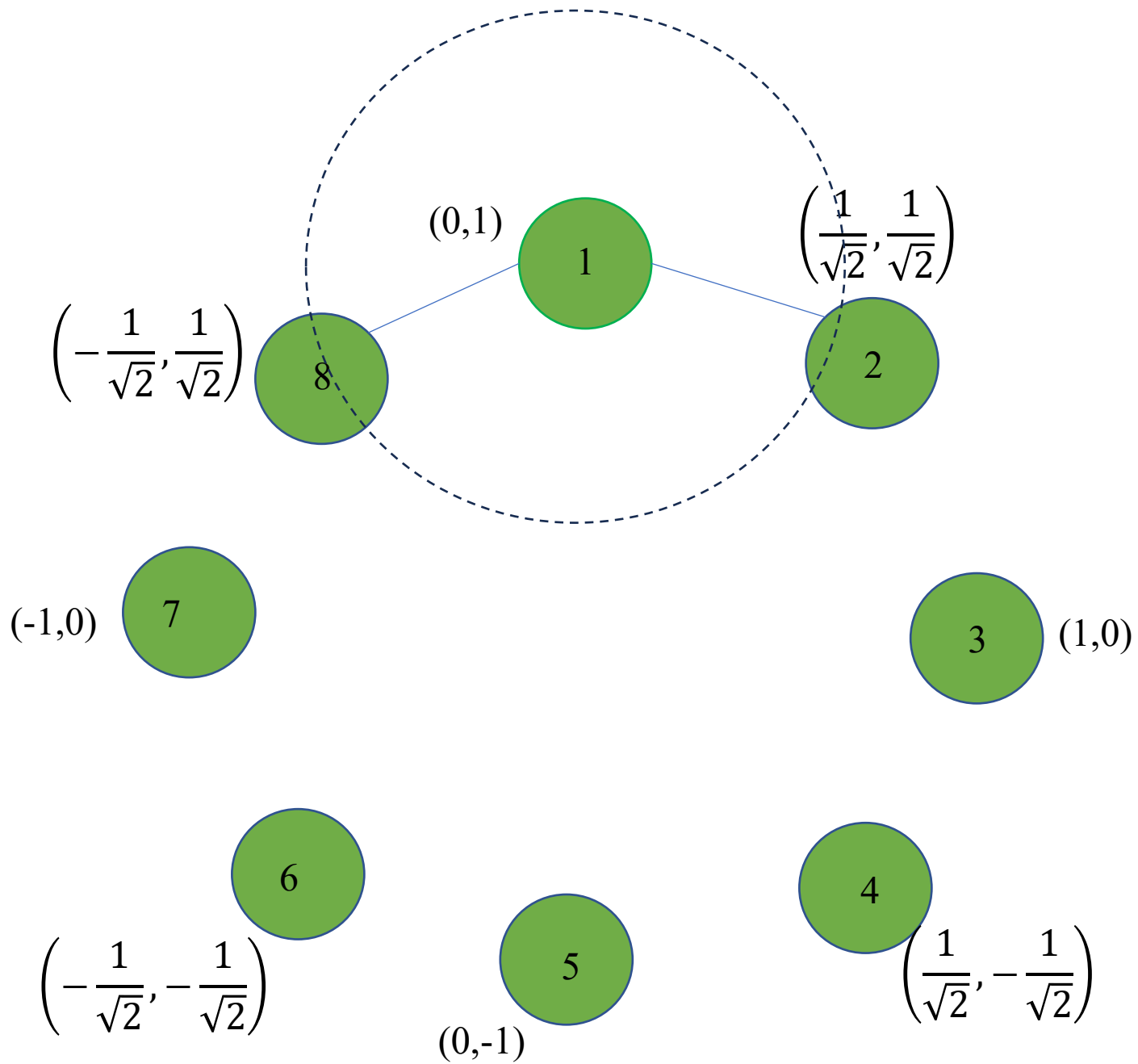


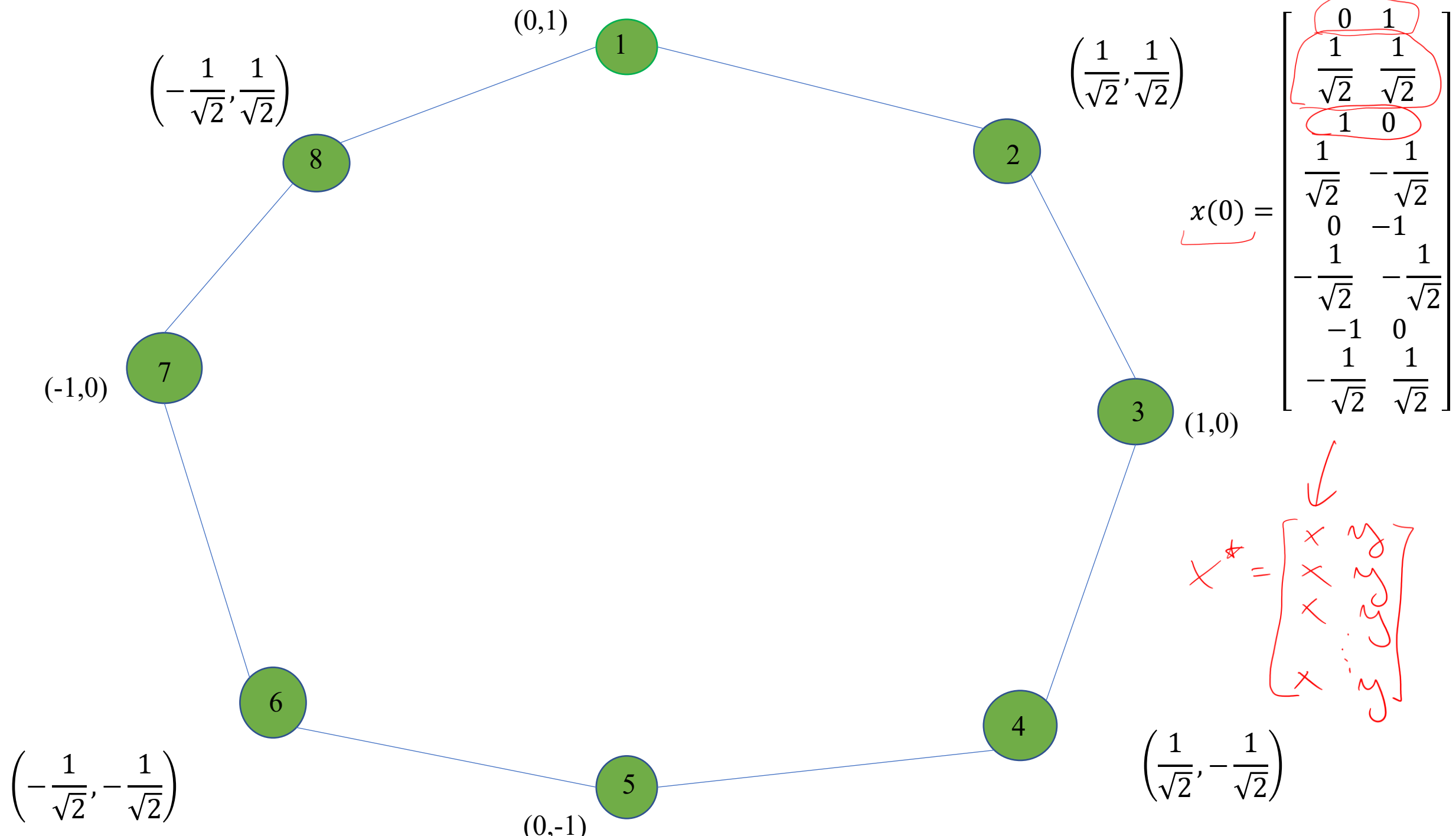


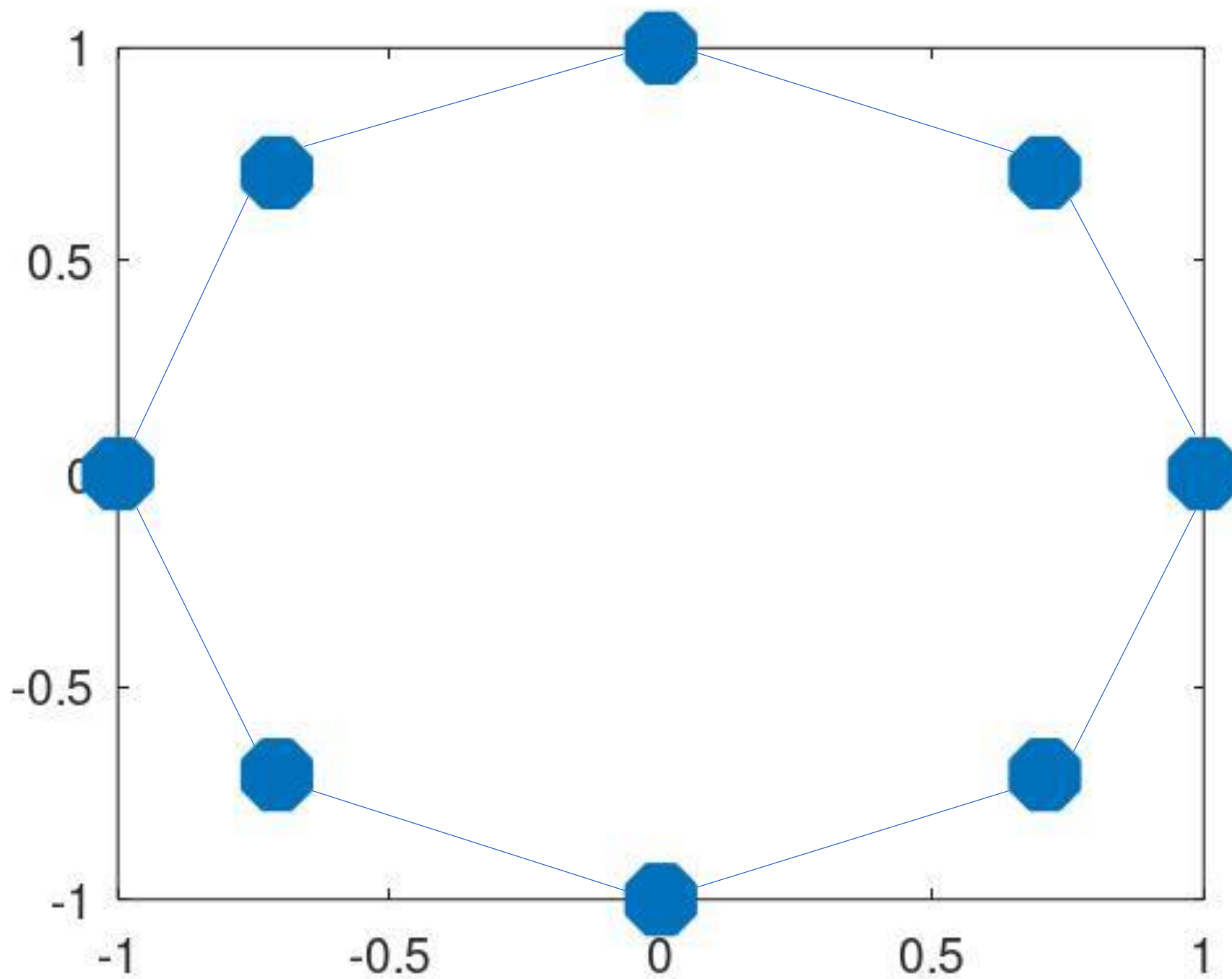
$$\left\| \begin{bmatrix} 0 \\ 1 \end{bmatrix} - \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} \right\| = \left\| \begin{bmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} \right\| = 1$$

Iterație Nod 1:

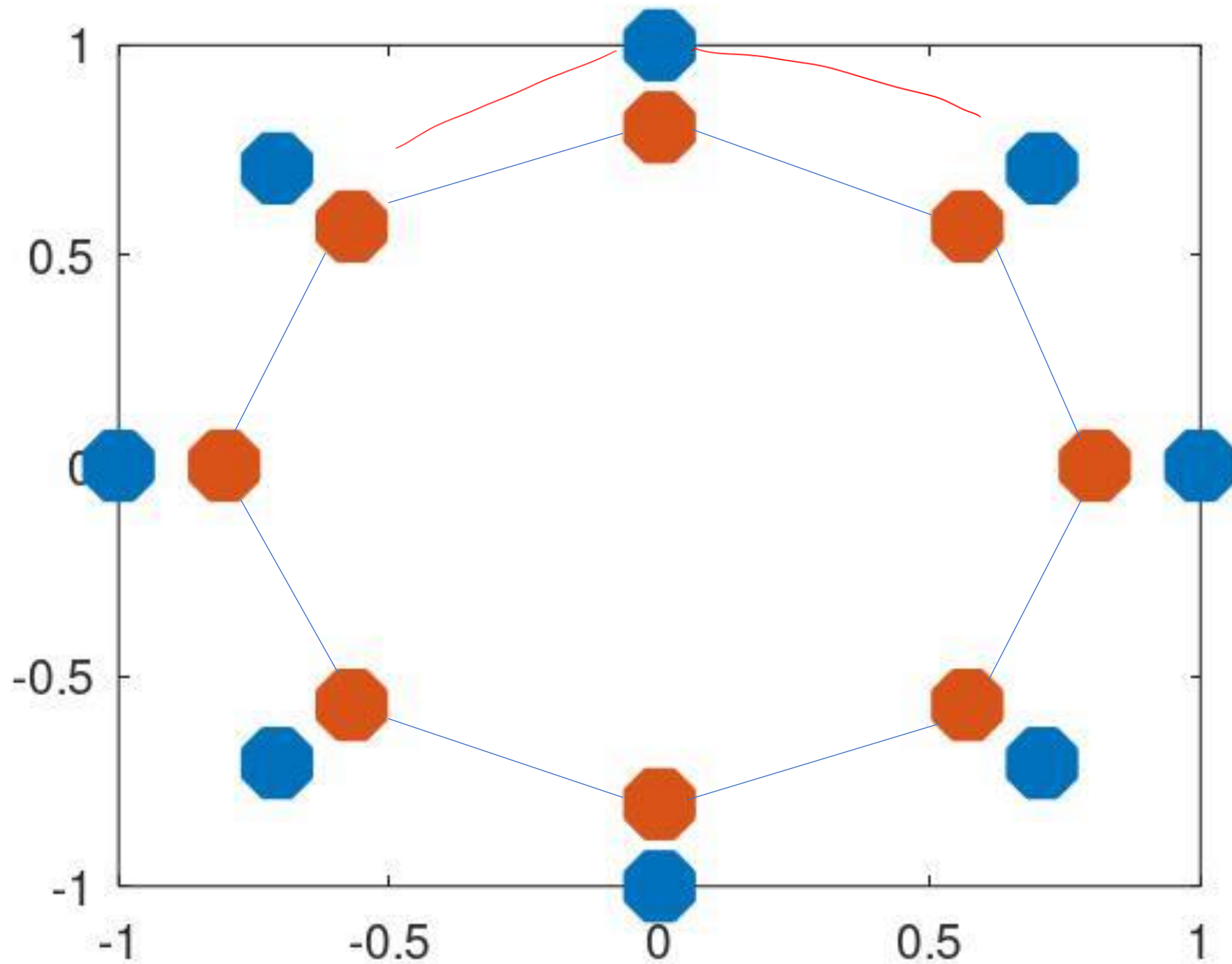
1. $\text{send}(p, \{\underline{1}, 8\});$
2. $\text{recv_p} = \text{recv}(\{\underline{1}, 8\});$
3. $p = p + \text{avrg}(\{p\} \cup \{\text{recv_p}\})$





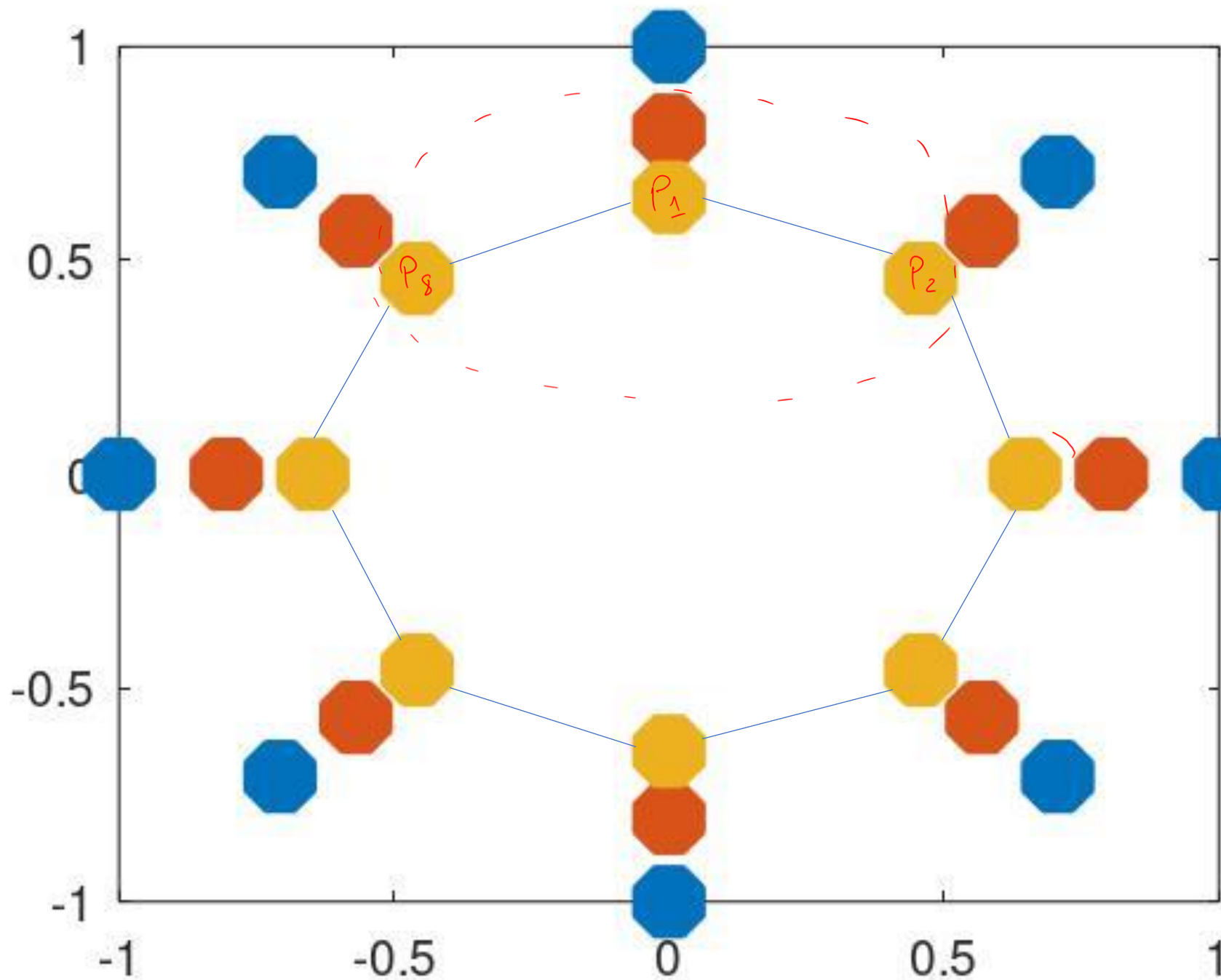


$$x(0) = \begin{bmatrix} 0 & 1 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 1 & 0 \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ 0 & -1 \\ -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ -1 & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix}$$

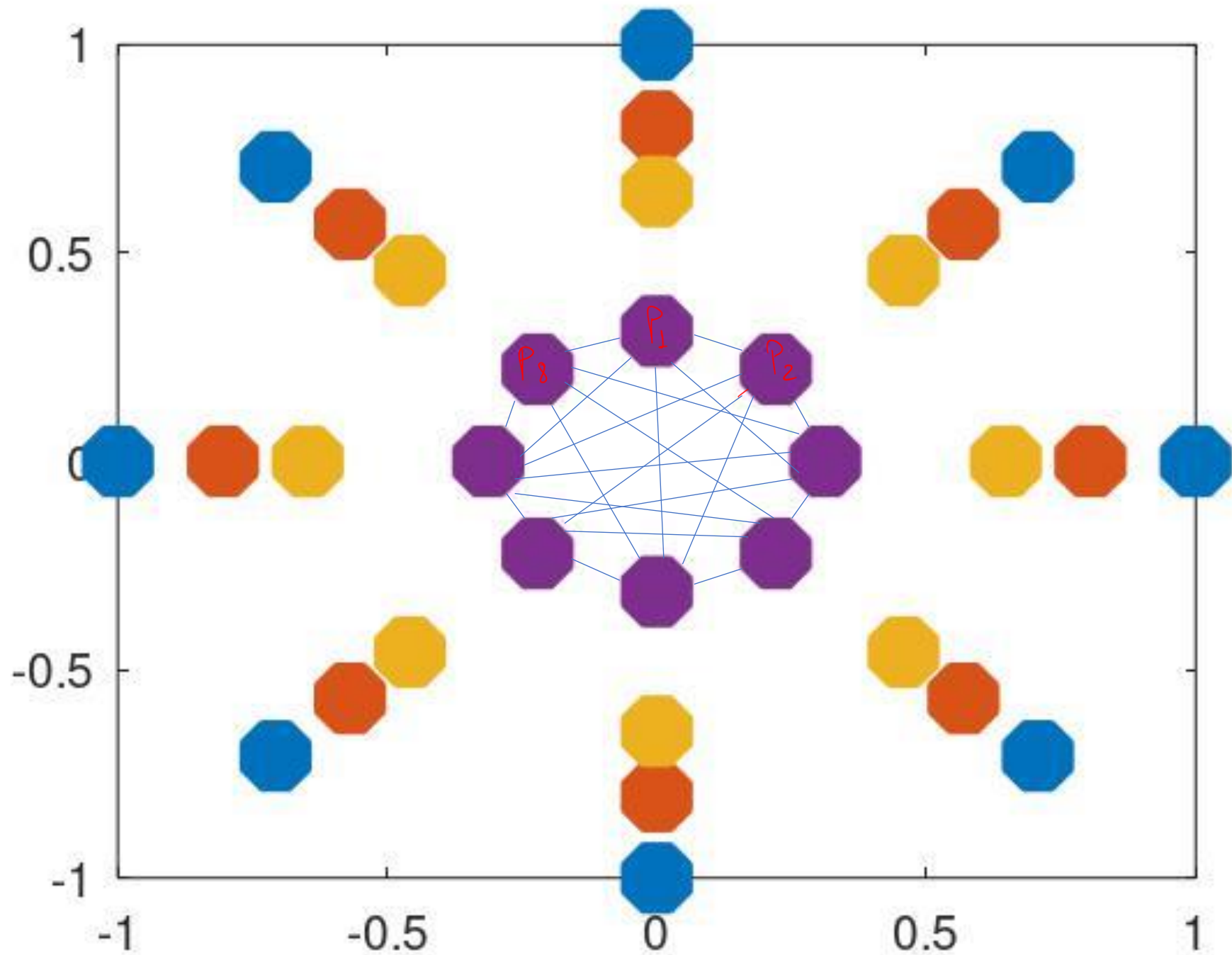


$$x(1) = \begin{bmatrix} 0 & 0.8047 \\ 0.5690 & 0.5690 \\ 0.8047 & 0 \\ 0.5690 & -0.5690 \\ 0 & -0.8047 \\ -0.5690 & -0.5690 \\ -0.8047 & 0 \\ -0.5690 & 0.5690 \end{bmatrix}$$

$$\sum_{i,j} \|x(1)[i,:] - x(1)[j,:]\|$$



$$x(2) = \begin{bmatrix} P_1 & 0 & 0.6476 \\ P_2 & 0.4579 & 0.4579 \\ & 0.6476 & 0 \\ & 0.4579 & -0.4579 \\ & 0 & -0.6476 \\ & -0.4579 & -0.4579 \\ & -0.6476 & 0 \\ P_8 & -0.4579 & 0.4579 \end{bmatrix}$$

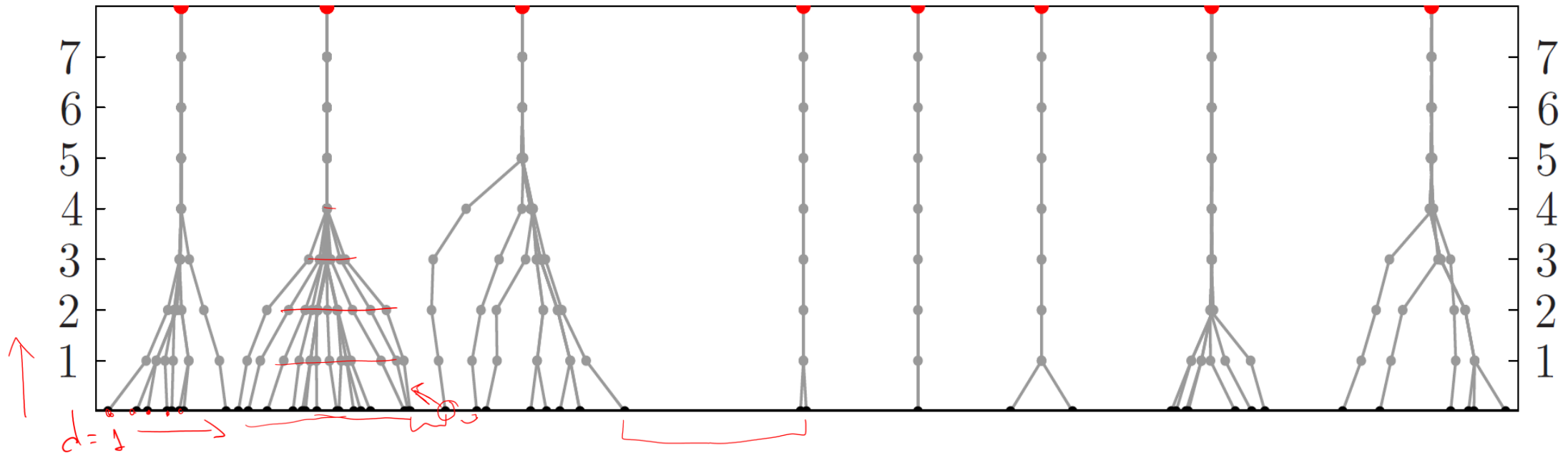


$$\underline{x(3)} = \begin{bmatrix} 0 & 0.3127 \\ 0.2211 & 0.2211 \\ 0.3127 & 0 \\ 0.2211 & -0.2211 \\ 0 & -0.3127 \\ -0.2211 & -0.2211 \\ -0.3127 & 0 \\ -0.2211 & 0.2211 \end{bmatrix}$$

$$\|x^{(3)}(i,:) - x^{(3)}(j,:)\| \leq 1$$

$(i,j) \text{ vicini}$

Studiu de caz (graf neconectat)

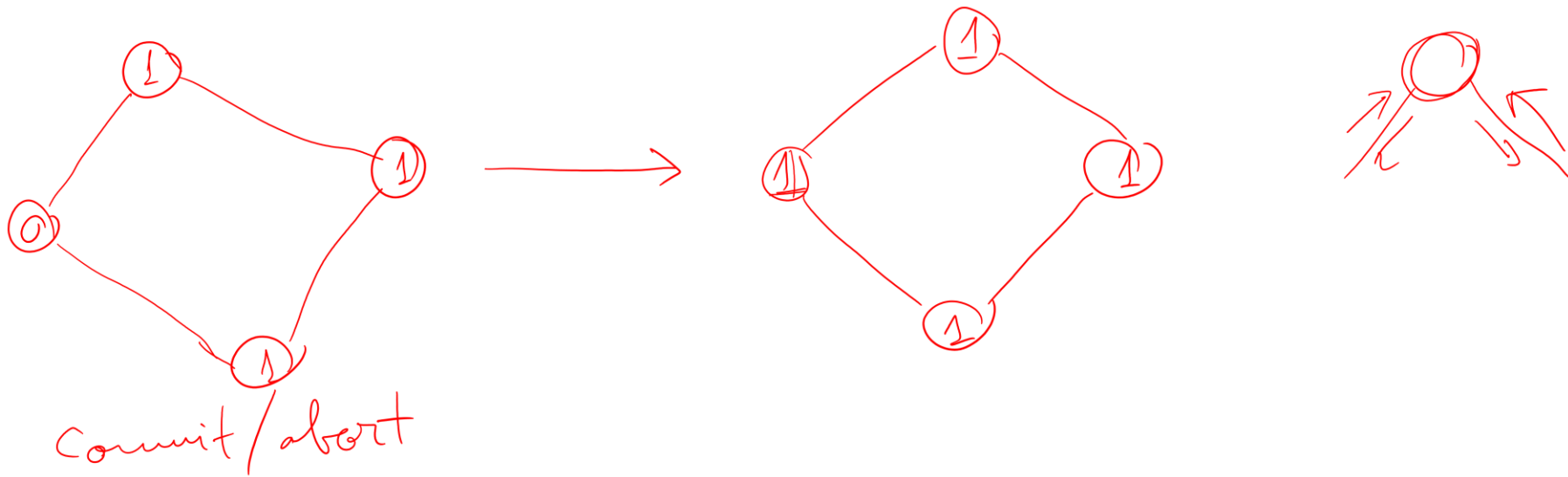


- Axa Oy este contorul iterațiilor, axa Ox pozițiile agenților (valori reale) distribuiți aleator $[-15, 15]$
- Graful inițial nu este conex (consensul este imposibil!)
- Algoritmul de Medie este implementat pentru $r = 1.5$ (maxim 7 iterații)
- Rezultat: agenții se grupează și converg la un *rendezvous* (chiar dacă nu unul comun); *în unele cazuri, agenții sunt conectați la început și deconectați la final.*
- Modificări ale algoritmului de medie exclude aceste cazuri.

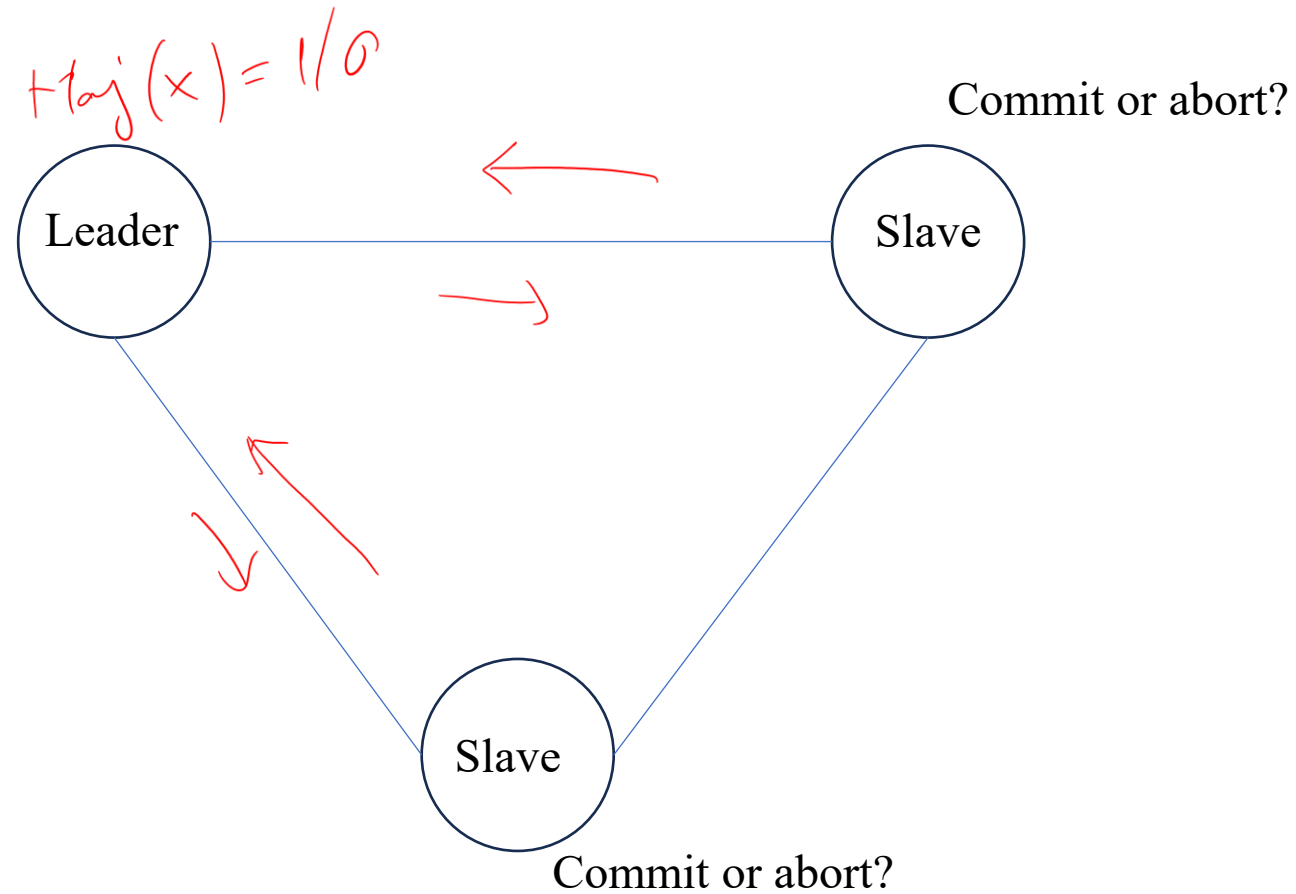
Vot majoritar

Problema votului

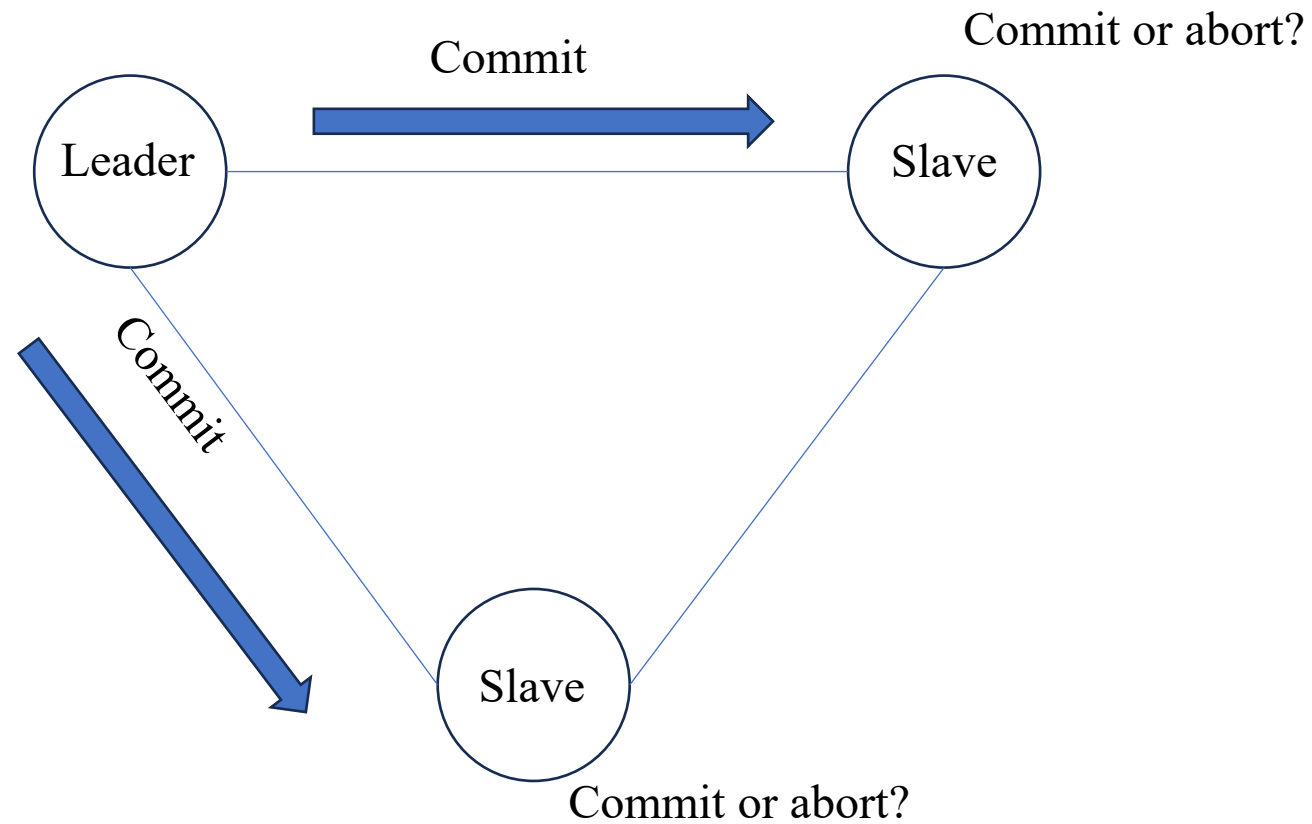
Problema votului majoritar. Un agent dispune de 2 valori posibile ale stării sale (1/0) pentru a decide (vota) local asupra unei chestiuni globale. Pornind de la voturile agenților, realizați distribuit o procedură de evaluare a majorității și acordul între agenți asupra deciziei majorității.



Problema votului centralizat



Problema votului centralizat



Problema votului centralizat

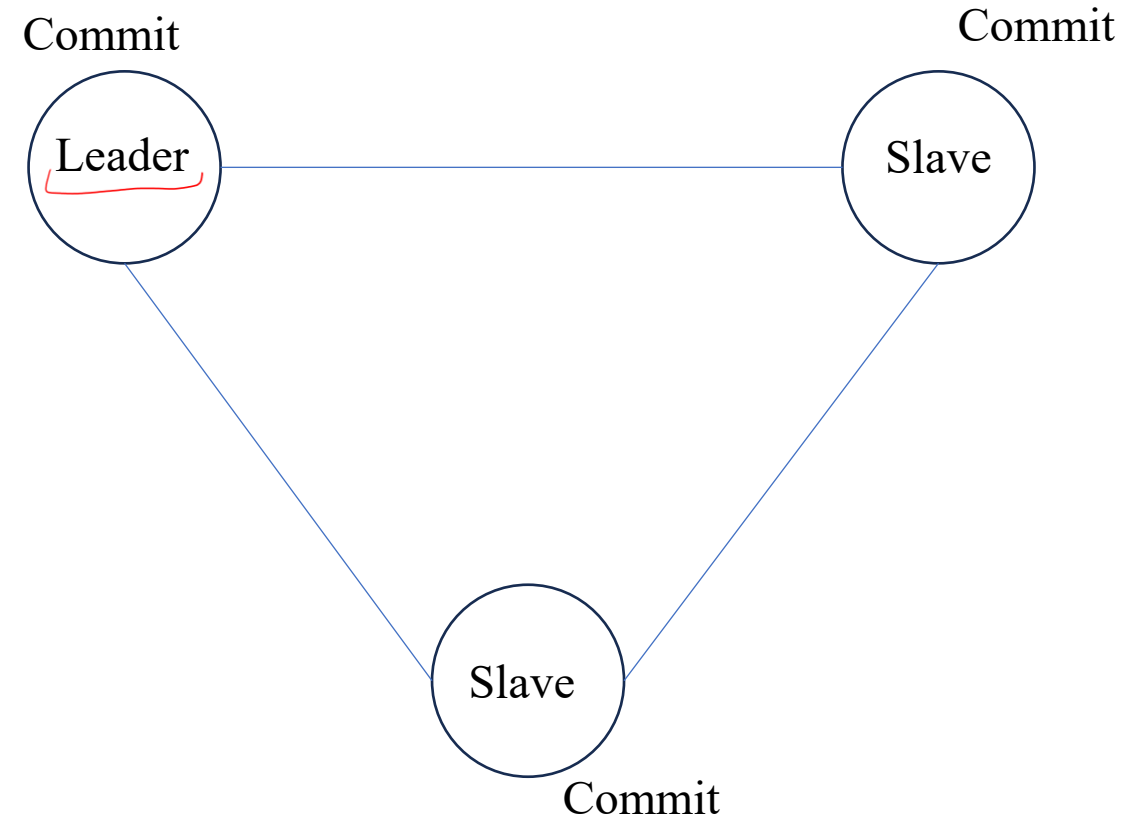
În context sincron fără defecte, asigurarea consensului centralizat se realizează printr-o simplă difuzare de mesaje.

Dificultatea rămâne selecția preliminară a liderului, care se realizează folosind algoritmi sincroni AL.

Vot binar majoritar (centralizat)

- stare lider x_l

1. $\text{buf} = \text{Gather}(G)$; ←
2. $x_l = \begin{cases} 1, \text{dacă } |\{\text{buf}_i = 1\}| \geq \frac{n}{2} + 1 \\ 0, \text{dacă } |\{\text{buf}_i = 1\}| < \frac{n}{2} + 1 \end{cases}$ }
3. $\text{Broadcast}(x_l)$; ←



Problema votului centralizat

Vot binar majoritar (centralizat)

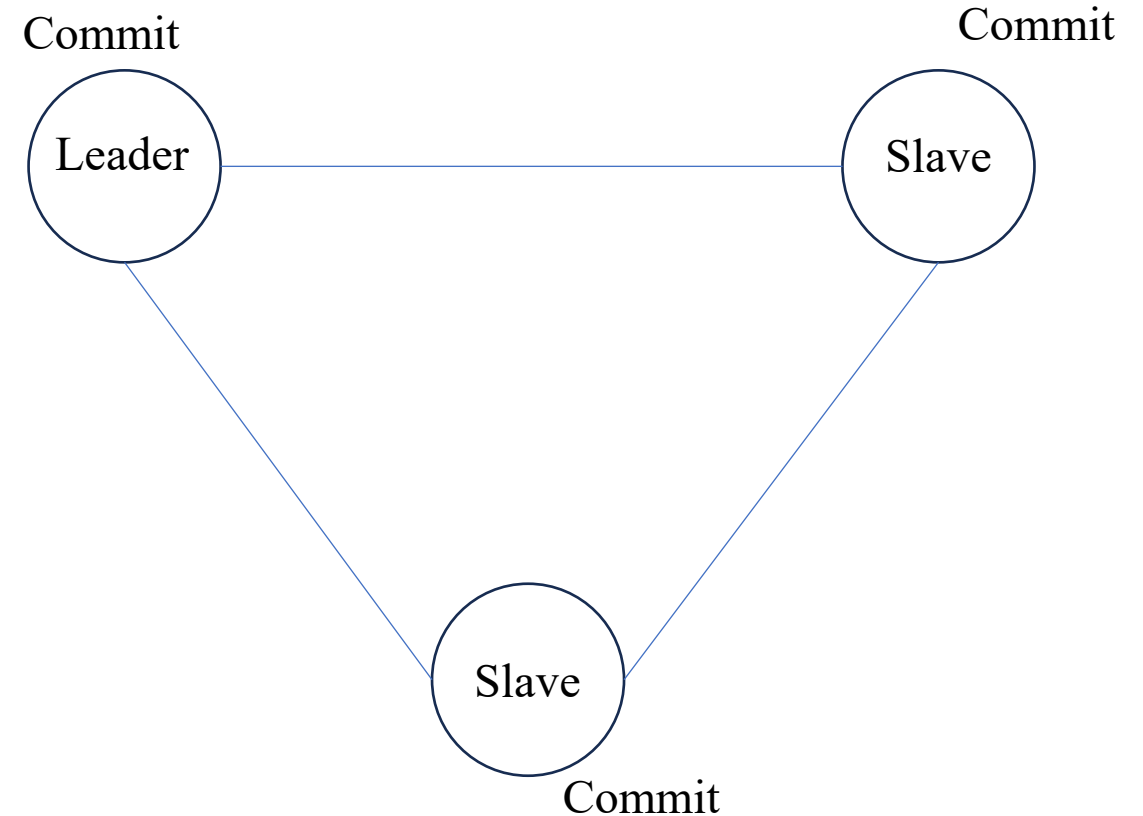
- stare lider $x_l \in \{0,1\}$

1. $\text{buf} = \text{Gather}(\underline{G})$;

2. $x_l = \begin{cases} 1, \text{dacă } |\{buf_i = 1\}| \geq \left\lceil \frac{n}{2} \right\rceil \\ 0, \text{dacă } |\{buf_i = 1\}| < \left\lceil \frac{n}{2} \right\rceil \end{cases}$

3. $\text{Broadcast}(x_l)$;

- Slave: stochează 1 bit cu decizia curentă.
- Dacă avem perturbații pe noduri sau pe legături, schema de mai sus nu funcționează (rezultat posibil greșit).
- De asemenea, un lider defect impune reluarea procedurii de AL distribuit.



Problema votului distribuit

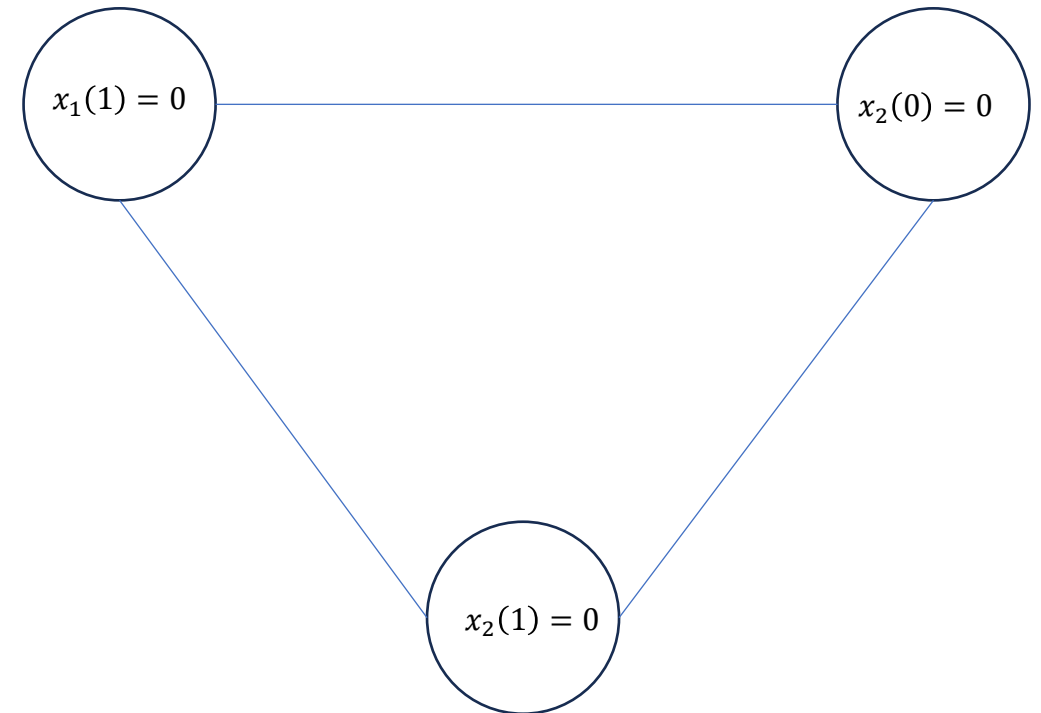
Vot binar majoritar (distribuit)

- stare lider $x_i \in \{0,1\}$

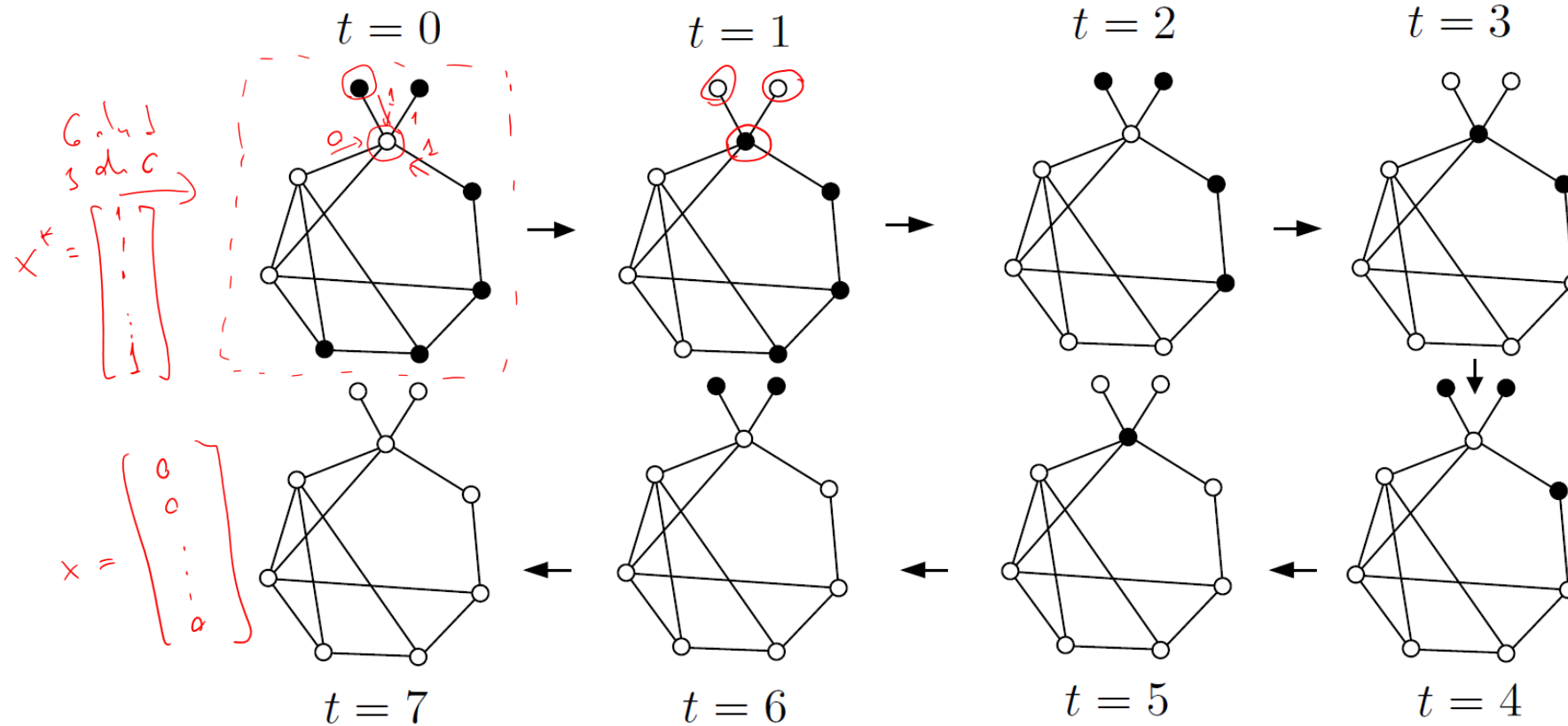
1. $buf_i = \text{Gather}(\mathcal{N}_i);$
2. $x_i = \begin{cases} 1, & \text{dacă } |\{buf_i[j] = 1\}| \geq \left\lceil \frac{|\mathcal{N}_i|}{2} \right\rceil \\ 0, & \text{dacă } |\{buf_i[j] = 1\}| < \left\lceil \frac{|\mathcal{N}_i|}{2} \right\rceil \end{cases}$
3. $\text{Broadcast}(x_i, \mathcal{N}_i);$

Actualizări locale bazate pe calculul „majorității” valorilor provenite de la vecini.

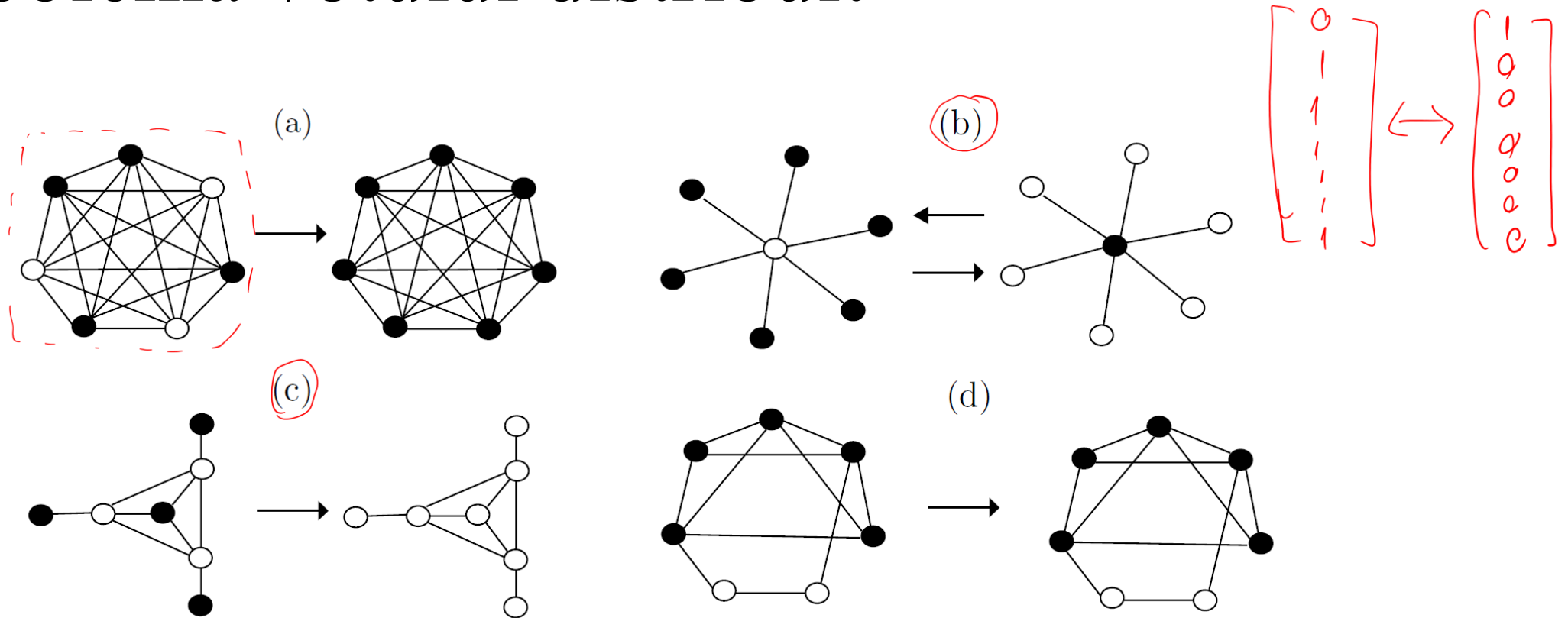
Alte topologii potrivite/nepotrivite?



Problema votului distribuit



Problema votului distribuit



- Majority consensus computer = o topologie care permite calcularea distribuită (prin actualizări locale) a votului majoritar.
- „...the full characterization of m.c.c.’s remains an open problem”

Problema votului distribuit

Teoremă de imposibilitate [Land & Belew]. Fie sistemul $(\{x(t)\}_{t \geq 0}, \mathcal{G})$ cu n noduri și stări binare $x(t) \in \{0,1\}^n$. Nu există un algoritm determinist, sincron, distribuit care rezolvă exact *problema de consens binar majoritar* (pentru oricare $\mathcal{G}$).

Concluzie: Numărul (natura) stărilor per nod este un factor important în rezolvarea distribuită a problemelor centralizate.

Problema votului distribuit

Teoremă de imposibilitate [Land & Belew]. Fie sistemul $(\{x(t)\}_{t \geq 0}, \mathcal{G})$ cu n noduri și stări binare $x(t) \in \{0,1\}^n$. Nu există un algoritm determinist, sincron, distribuit care rezolvă exact *problema de consens binar majoritar* (pentru oricare $\mathcal{G}$).

Cum depășim teorema de imposibilitate?

- Stări multiple (e.g. reale, nu binare), $B > 1$
- Automate probabilistice
- Automate asincrone

Calcul majoritate binară

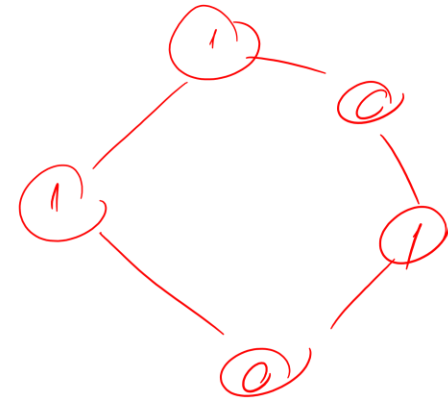
Dacă stările sunt numere reale (nu binare) cum calculăm valoarea majoritară a unei stări inițiale binare?

$$x_i(t) \in \mathbb{R} \mid x_i(0) \in \{0, 1\}$$

$$x_i^* = \text{Majority}(x(0)) = \begin{cases} 1, & |i \mid x_i(0) = 1 \mid \geq n/2 \\ 0, & |i \mid x_i(0) = 0 \mid \geq n/2 \end{cases}$$

$$x(0) = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{pmatrix} \Rightarrow \text{Avg}(x(0)) = \frac{3}{5} \geq \frac{1}{2} \Rightarrow \text{tag} = 1$$
$$< \frac{1}{2} \Rightarrow \text{tag} = 0$$

$$\text{Majority}(x) = \frac{1}{2} \left[\text{sgn} \left(\text{Avg}(x) - \frac{1}{2} \right) + 1 \right]$$



Un posibil algoritm?

$$\text{Avg}(x(t)) = \frac{1}{n} (x_1(t) + x_2(t) + \dots + x_n(t))$$

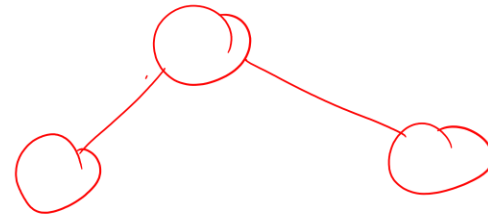
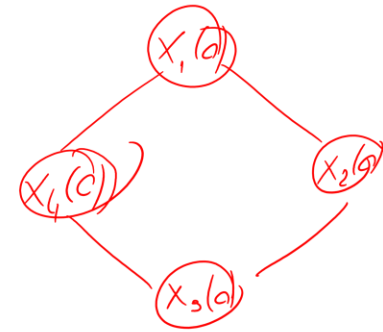
1. Recv(x_v , vicini)
2. $x(t+1) = \text{avg}(\{x_v, x(t)\})$
3. send($x(t+1)$, vicini)

$$x_v = \begin{cases} x_{\text{str}} & i-1 \bmod n \\ x_{\text{dr}} & i+1 \bmod n \end{cases}$$

$$x_i(t+1) = \frac{1}{3} (x_i(t) + x_{\text{str}}(t) + x_{\text{dr}}(t))$$

$$= \frac{1}{3} x_{\text{str}}(t) + \frac{1}{3} x_i(t) + \frac{1}{3} x_{\text{dr}}(t) \quad \forall i$$

$$\begin{cases} x_1(t+1) = \frac{1}{3} x_n(t) + \frac{1}{3} x_1(t) + \frac{1}{3} x_2(t) \\ x_2(t+1) = \frac{1}{3} x_1(t) + \frac{1}{3} x_2(t) + \frac{1}{3} x_3(t) \\ \vdots \\ x_n(t+1) = \frac{1}{3} x_{n-1}(t) + \frac{1}{3} x_n(t) + \frac{1}{3} x_1(t) \end{cases} \Rightarrow x(t+1) = A \cdot x(t)$$



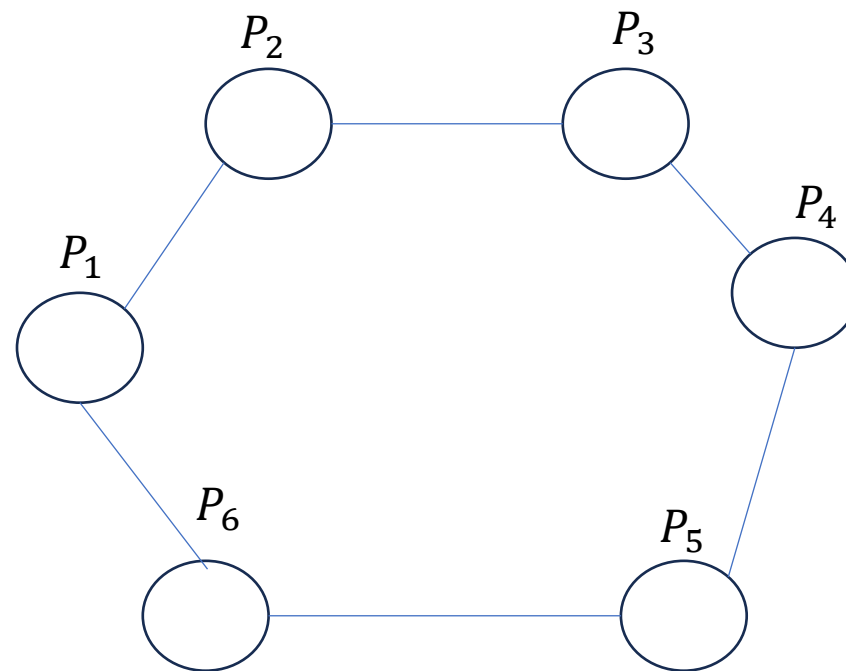
$$\frac{L \in \max\{v_1, v_2, \dots, v_n\}}{\max\{v_i, v_j, v_{\text{propru}}\}}$$

Topologia inel

Pentru un inel de dimensiune $n = 6$, matricea asociată este:

$$A = \begin{bmatrix} 1/3 & 1/3 & 0 & 0 & 0 & 1/3 \\ 1/3 & 1/3 & 1/3 & 0 & 0 & 0 \\ 0 & 1/3 & 1/3 & 1/3 & 0 & 0 \\ 0 & 0 & 1/3 & 1/3 & 1/3 & 0 \\ 0 & 0 & 0 & 1/3 & 1/3 & 1/3 \\ 1/3 & 0 & 0 & 0 & 1/3 & 1/3 \end{bmatrix}$$

dublu stohastica



- Inițial: $x_j(0) = v_j \in R$
- Iterație locală: $x_i(t + 1) = \frac{1}{d_i + 1} \left(x_i(t) + \sum_{j \in \mathcal{N}_i^-} x_j(t) \right), \quad \forall i$
- Analiza

Algoritm Flooding pentru consens majoritar

Iterație locală: $x_i(t+1) = \frac{1}{d_i+1} (x_i(t) + \sum_{j \in \mathcal{N}_i^-} x_j(t))$, $\forall i$

$$V(x) = \max_{i,j} x_i - x_j \quad | \quad \boxed{V(x)=0} \Leftrightarrow x_i = x_j, \forall i,j$$

$$\begin{aligned} \underline{V(x(t+1))} &= \max_{i,j} x_i(t+1) - x_j(t+1) = \max_{i,j} \underbrace{A_i \cdot x(t)} - \underbrace{A_j \cdot x(t)} \\ &= \max_{i,j} \frac{1}{3} (x_{i_1}(t) + x_{i_2}(t) + x_{i_3}(t)) - \frac{1}{3} (x_{j_1}(t) + x_{j_2}(t) + x_{j_3}(t)) \\ &= \max_{i,j} \frac{1}{3} [(x_{i_1}(t) - x_{j_1}(t)) + (x_{i_2}(t) - x_{j_2}(t)) + (x_{i_3}(t) - x_{j_3}(t))] \\ &\leq \frac{1}{3} \left[\underbrace{\max_{i,j} x_i(t) - x_j(t)}_{V(x(t))} + \underbrace{\max_{i,j} x_i(t) - x_j(t)}_{V(x(t))} + \underbrace{\max_{i,j} x_i(t) - x_j(t)}_{V(x(t))} \right] \\ &= V(x(t)) \Rightarrow V(x(t+1)) \leq V(x(t)), \forall t. \end{aligned}$$

Algoritm Flooding cu ponderi uniforme

Algoritm **Flooding**(Maj, v):

Mem_i : - int x_i (token) , inițial v_i
- int d (grad intrare), integer
- int t , integer, inițial 0

- SIP7D
- info locală de la N_i
- rezolvă problema votul (pe inel)

Funcție transformare nod i ():

1. Fie U mulțimea mesajelor $v_j = x_j(t)$ primite de la N_i^-
2. $x_i(t+1) = \frac{1}{d+1} \left(x_i(t) + \sum_{j \in N_i^-} x_j(t) \right)$
3. **If** (criteriu_oprire):
 1. **Return** $\frac{1}{2} \left(1 + \text{sgn} \left(x(t) - \frac{1}{2} \right) \right)$
4. **Else**: send($x(t+1)$, N_i^+)
5. $t := t+1$